

Experiment no __

Aim : Implement ceaser cipher & simple columnar techniques.

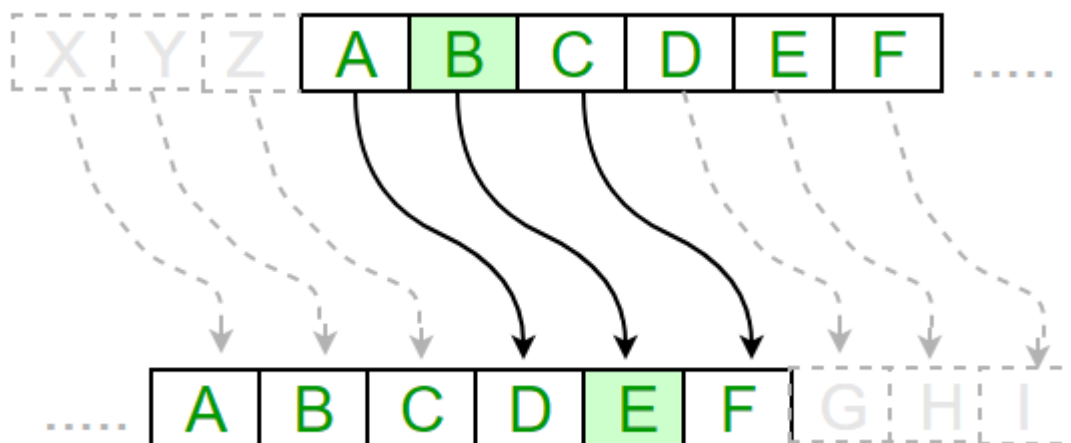
Description / Theory :

1. Ceaser cipher

The Caesar Cipher technique is one of the earliest and simplest methods of encryption technique. It's simply a type of substitution cipher, i.e., each letter of a given text is replaced by a letter with a fixed number of positions down the alphabet. For example with a shift of 1, A would be replaced by B, B would become C, and so on. The method is apparently named after Julius Caesar, who apparently used it to communicate with his officials.

Thus to cipher a given text we need an integer value, known as a shift which indicates the number of positions each letter of the text has been moved down.

The encryption can be represented using modular arithmetic by first transforming the letters into numbers, according to the scheme, A = 0, B = 1, ..., Z = 25. Encryption of a letter by a shift n can be described mathematically as.



Examples :

Text : ABCDEFGHIJKLMNOPQRSTUVWXYZ

Shift: 23

Cipher: XYZABCDEFGHJKLMNOPQRSTUVWXYZ

Text : ATTACKATONCE

Shift: 4

Cipher: EXXEGOEXSRGI

How to decrypt?

We can either write another function decrypt similar to encrypt, that'll apply the given shift in the opposite direction to decrypt the original text. However we can use the cyclic property of the cipher under modulo, hence we can simply observe

$$\text{Cipher}(n) = \text{De-cipher}(26-n)$$

Hence, we can use the same function to decrypt, instead, we'll modify the shift value such that $\text{shift} = 26 - \text{shift}$

Program :

```
import java.util.Scanner;
```

```
public class CeaserCipherAlgo{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String alphabets = "abcdefghijklmnopqrstuvwxyz";
        String cipherText = "";
        String originalText = "";

        System.out.println("Enter the text to encrypt : ");
        String txt = sc.nextLine();
        System.out.println("Enter the number of shift : ");
        int shift = sc.nextInt();

        // converting to lowercase
        txt = txt.toLowerCase();
        // Encryption //
        // System.out.println(alphabets.indexOf(charAt(i)));
        for(int i = 0; i < txt.length(); i++)
```

```

{
    // Position of character of txt in alphabet
    int charPosition = alphabets.indexOf(txt.charAt(i));

    // Increasing by number of the shift given by user
    int keyVal = (shift + charPosition)%26;

    // Replace character by shifted value of original text
    char replaceVal = alphabets.charAt(keyVal);

    // cipherText = cipherText + replaceVal
    cipherText += replaceVal;
}
System.out.println("\nEncrypted text : "+cipherText);

// Decryption //
for(int i = 0; i<cipherText.length(); i++)
{
    //position of character of decrypted (text)
    int charPosition = alphabets.indexOf(cipherText.charAt(i));

    // Decreasing by number of shift
    int keyVal = (charPosition - shift)%26;

    //Replace character by shifted value
    char replaceVal2 = alphabets.charAt(keyVal);

    // originalText = originalText + replaceVal2
    originalText += replaceVal2;
}
System.out.println("\nDecrypted text : "+originalText);
sc.close();
}
}

```

Output :

Enter the text to encrypt :
 Akash
 Enter the number of shift :
 3

Encrypted text : dndvk

Decrypted text : akash

2. Simple Columnar

A simple example for a transposition cipher is **columnar transposition cipher** where each character in the plain text is written horizontally with specified alphabet width. The cipher is written vertically, which creates an entirely different cipher text.

Consider the plain text **hello world**, and let us apply the simple columnar transposition technique as shown below

h	e	l	l
o	w	o	r
l	d		

The plain text characters are placed horizontally and the cipher text is created with vertical format as : **holewdlo lr**. Now, the receiver has to use the same table to decrypt the cipher text to plain text.

Program :

```
import java.util.ArrayList;
import java.util.Scanner;

class simpleColumnarAlgo2 {
    public static void main(String args[])
    {
        Scanner input = new Scanner(System.in);

        //inputs
```

```

System.out.print("\nEnter a text to encrypt :");
String txt = input.nextLine();
System.out.print("Enter number of column(size) : ");
int size = input.nextInt();
System.out.print("Enter order to encrypt :");
int[] order = new int[size];

for (int i = 0; i < size; i++)
{
    order[i] = input.nextInt();
}

printMatrix(txt, size);
String caesarText = convert(txt, size, order);
System.out.println("\n\tCaesar text : " + caesarText);
String plainText = decrypt(caesarText, order, size);
System.out.println("\n\tPlain text : " + plainText);
input.close();
}

public static String convert(String msg, int colSize, int[] order)
{
    msg = msg.replaceAll("\\s", "").toUpperCase();
    ArrayList<Character> arr = new ArrayList<Character>();
    for (int i = 0; i < msg.length(); i++)
    {
        arr.add(msg.charAt(i));
    }
    char[] res = new char[arr.size()];
    for (int i = 0; i < arr.size(); i++)
    {
        int n = 0;
        while (n < colSize)
        {
            int x = order[n];
            int a = x - 1;
            for (int j = 0; j < Math.ceil(arr.size() / colSize) + 1; j++)
            {
                if (a < arr.size())

```

```

        {
            res[i] = arr.get(a);
            i++;
            a += colSize;
        }
    }
    n++;
}
}
return String.valueOf(res);
}

```

```

static String decrypt(String msg, int[] order, int colSize) {
    ArrayList<Character> arr = new ArrayList<Character>();
    char[][] temp = new char[(int) Math.ceil((double)msg.length() / (double)
colSize)][colSize];
    char[] result = new char[msg.length()];
    for (int i = 0; i < msg.length(); i++)
    {
        arr.add(msg.charAt(i));
    }
    int noElementsInLastRow = msg.length() % colSize;
    int i = 0;
    while (i < arr.size())
    {
        for (int k = 0; k < order.length; k++)
        {
            int a = order[k] - 1;
            if (a < noElementsInLastRow)
            {
                for (int j = 0; j < Math.ceil((double)msg.length() / (double) colSize);
j++)
                {
                    temp[j][a] = arr.get(i); i++;
                }
            } else {
                for (int j = 0; j < Math.floor((double)msg.length() / (double) colSize);
j++)

```

```

        {
            temp[j][a] = arr.get(i);i++;
        }
    }
}
}
int m = 0;
for (int j = 0; j < Math.ceil((double) msg.length() / (double) colSize); j++){
    for (int k = 0; k < colSize; k++)
    {
        if(m < result.length)
        {
            result[m] = temp[j][k];m++;
        }
    }
}
return String.valueOf(result);
}
static void printMatrix(String msg, int colSize)
{
    msg = msg.replaceAll("\\s", "").toUpperCase();
    for (int i = 0; i < msg.length(); i++)
    {
        if (i % colSize == 0)
        {
            System.out.println();
        }
        System.out.print("\t" + msg.charAt(i) + "\t");
    }
    System.out.println();
}
}
}

```

Output :

```
Enter a text to encrypt :Akash
Enter number of column(size) : 3
Enter order to encrypt :2
1
3
```

A		K		A	
S		H			

Caesar text : KHASA

Plain text : AKASH

Conclusion : Thus, in this experiment we have study and successfully perform ceaser cipher and simple columnar algorithms.